

Manual de Estilo y Accesibilidad (ARIA)

Guia Oficial para la Creacion de Componentes en AVFenix UI

Este manual establece los estandares de arquitectura, convenciones visuales y requerimientos estrictos de accesibilidad (WCAG 2.1 / WAI-ARIA) para el desarrollo de componentes reactivos reutilizables en AVFenix UI sobre General.JS.

1. Arquitectura y Principios de Diseno

Los componentes de AVFenix UI deben adherirse a los siguientes principios fundamentales:

- Reactividad Fina: Aprovecha Proxies recursivos y el Virtual DOM de reactive.general.js. Evita mutaciones directas del DOM.
- Estado Local Inmutable: Siempre utiliza `this.setState(...)` para desencadenar ciclos de re-renderizado eficientes.
- Static Hoisting: Separa estructuras JSX estaticas de valores dinamicos para permitir la optimizacion automatica del compilador AFXC.

2. Convenciones de Nomenclatura

- Archivos .avf: Utiliza PascalCase estricto (ej. Button.avf, Modal.avf, AutoForm.avf).
- CSS BEM: Aplica el prefijo avf- seguido del bloque, elemento y modificador. Ejemplos: avf-button, avf-button--primary, avf-modal__header.
- Componentes exportables: Todo componente publico debe incluir la palabra clave export (ej. export component Modal).

3. Anatomia de un Componente AVFenix

Estructura canonica recomendada para cualquier componente de la libreria:

```
export component CustomCard {
  state = { expanded: false };

  onMount() {
    console.log("CustomCard inicializado correctamente.");
  }

  toggle() {
    this.setState({ expanded: !this.state.expanded });
  }

  template(state) {
    return (
      <div class="avf-card">
        <button onClick={() => this.toggle()} class="avf-card__trigger">
          {this.props.title}
        </button>
        {state.expanded && (
          <div class="avf-card__content">{this.props.children}</div>
        )}
      </div>
    );
  }
}
```

}

4. Especificacion de Accesibilidad (ARIA) y Diseno Inclusivo

Para garantizar la conformidad con WCAG 2.1 Nivel AA, todos los componentes de AVFenix UI deben implementar las siguientes pautas ARIA obligatorias:

4.1 Semantica y Roles ARIA (WAI-ARIA 1.2)

- Elementos Interactivos: Cuando se utilicen contenedores personalizados (<div> o) como botones o controles, es OBLIGATORIO asignar el rol correspondiente (role="button", role="dialog", role="tablist", role="alert").

- Botones de Cierre e Iconos: Todo icono o boton decorativo debe incluir aria-hidden="true" o proveer un texto alternativo accesible con aria-label.

4.2 Atributos Dinamicos de Estado ARIA

Los componentes deben sincronizar el estado reactivo local con los atributos ARIA nativos:

- aria-expanded: Requerido en acordeones, menus desplegados y modales (true cuando esta abierto, false cuando esta cerrado).

- aria-invalid y aria-describedby: Requerido en componentes de formulario (TextInput, Select) para asociar mensajes de error al lector de pantalla.

- aria-live: Asignar aria-live="polite" o aria-live="assertive" en componentes de notificaciones (Toast) y alertas dinamicas.

4.3 Navegacion por Teclado y Control del Foco

- Indexacion: Todos los elementos interactivos deben poseer tabIndex={0} para ser alcanzables mediante la tecla Tab.

- Eventos de Teclado: Implementar listeners onKeyDown para responder a Enter y Space en controles personalizados, y Escape para cerrar modales/dialogos.

- Focus Trapping: Modales y Dialogos deben atrapar el foco mientras permanecen abiertos y devolver el foco al elemento disparador al cerrarse.

```
// Ejemplo de Componente Accesible (Modal.avf con ARIA)
export component Modal {
  state = { open: false };

  close() {
    this.setState({ open: false });
    if (this.props.triggerRef) this.props.triggerRef.focus();
  }

  handleKeyDown(e) {
    if (e.key === 'Escape') this.close();
  }

  template(state) {
    if (!state.open) return null;
    return (
      <div
        class="avf-modal-backdrop"
        role="dialog"
        aria-modal="true"
        aria-labelledby="modal-title"
        onKeyDown={(e) => this.handleKeyDown(e)}
      >
```

```
    <div class="avf-modal-content" tabIndex={-1}>
      <h2 id="modal-title">{this.props.title}</h2>
      <button
        aria-label="Cerrar dialogo"
        onClick={() => this.close()}
        class="avf-btn-close"
      >
        x
      </button>
      <div class="avf-modal-body">{this.props.children}</div>
    </div>
  </div>
);
}
```

5. Integracion y Distribucion

- Slots y props.children: Utiliza props.children para componer layouts flexibles.
- Soporte de Metadatos @ui: Los componentes de formulario deben consumir los metadatos provenientes del manifiesto .schema.json.
- Empaquetado NPM: Ejecuta npx avfenix ui:pack para auditar el componente y empaquetarlo para la distribucion oficial.